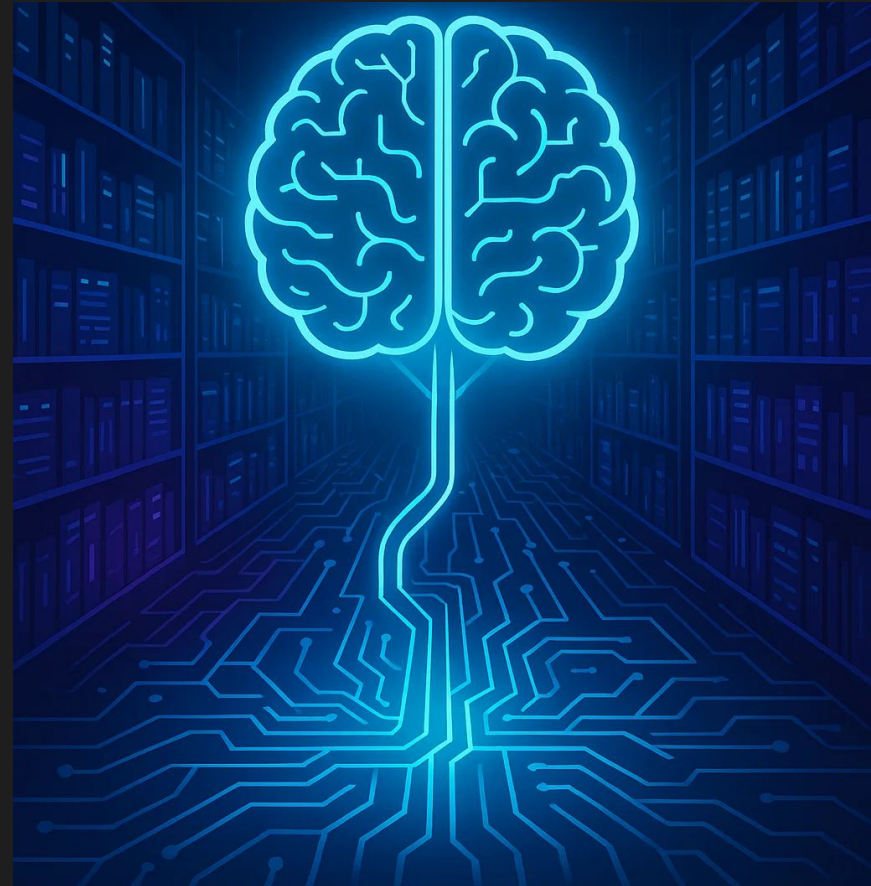


Unlocking Smarter AI: An Introduction to Retrieval-Augmented Generation (RAG)

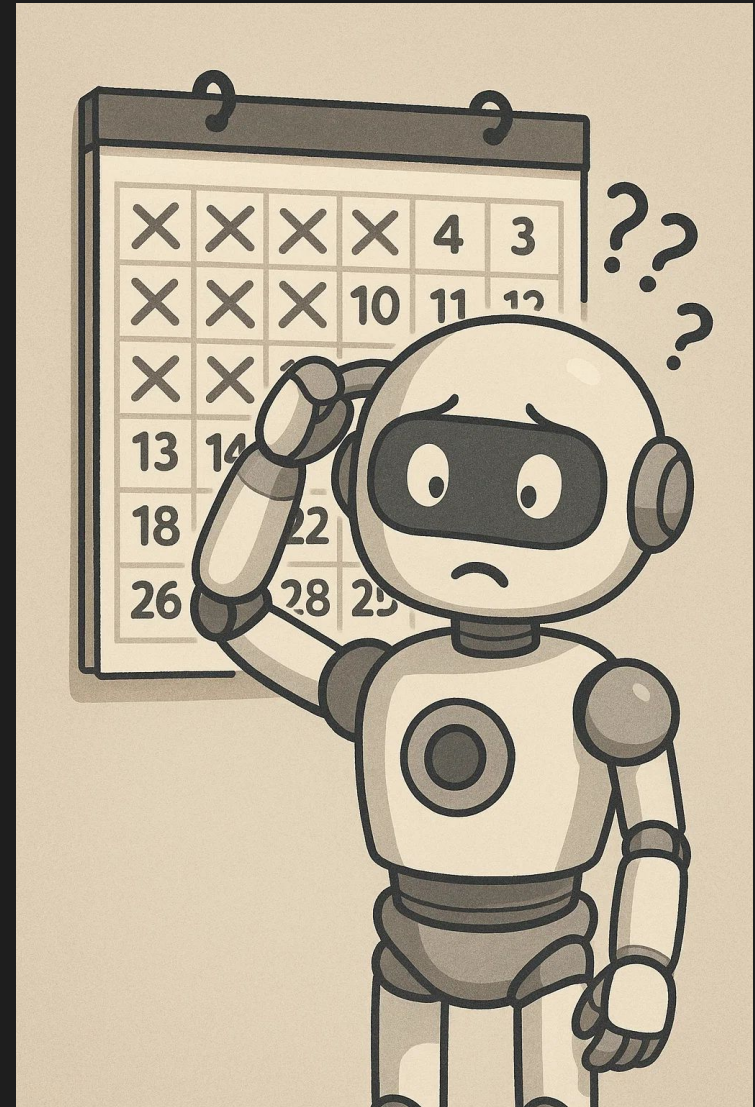
Enhancing Large Language Models with External Knowledge

Dr. Sri Phani Krishna Karri
EED, NIT Andhra Pradesh



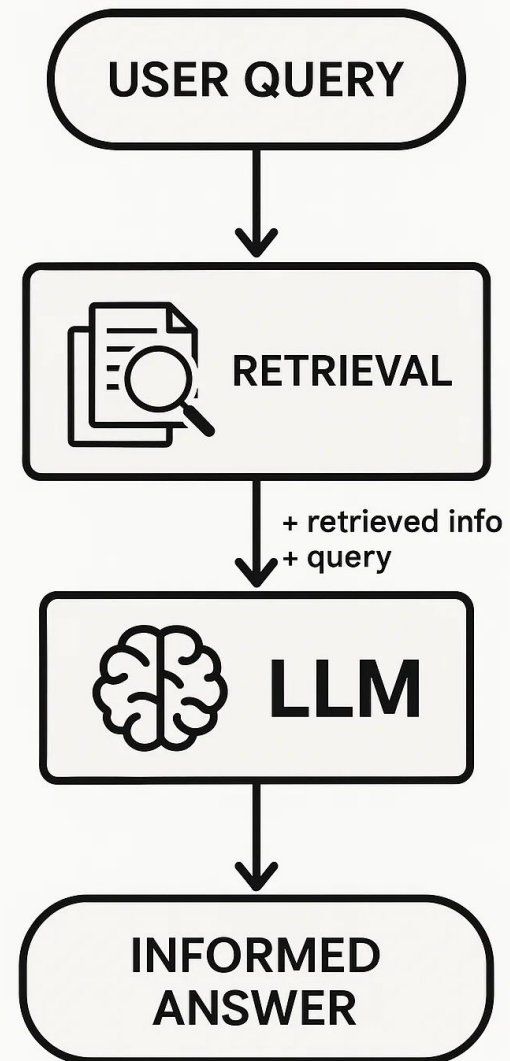
Why Do We Need RAG? Limitations of Standard LLMs

- LLMs are powerful but trained on static datasets.
- Knowledge "cut-off" point: Information becomes outdated.
- Potential for "hallucinations": Generating plausible but incorrect or fabricated information.
- Lack of domain-specific or real-time knowledge without costly retraining.
- Difficulty citing sources or explaining reasoning based on specific data.



What is Retrieval-Augmented Generation (RAG)?

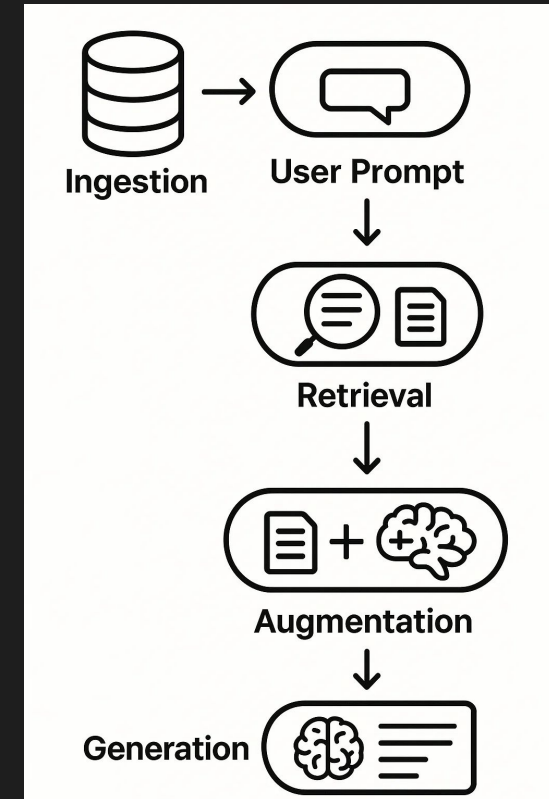
- An architecture enhancing LLMs by integrating external, up-to-date, and trustworthy data sources *before* generation.
- Combines the strengths of retrieval systems (finding relevant info) and generative models (creating fluent text).
- Goal: Provide LLMs with relevant context to generate more informed, accurate, and reliable responses.
- Analogy: An "open-book" exam for LLMs, allowing them to consult relevant materials.



How Does RAG Work? The Core Steps

1. **Ingestion (Prep):** Process external knowledge sources (documents, databases). Chunking & Embedding.
2. **User Prompt:** User asks a question.
3. **Retrieval:** System searches the external knowledge base for relevant information based on the prompt. This often involves converting the query to an embedding and finding similar document embeddings.
4. **Augmentation:** Retrieved information is added as context to the original prompt.
5. **Generation:** This augmented prompt (original query + retrieved context) is fed to the LLM.

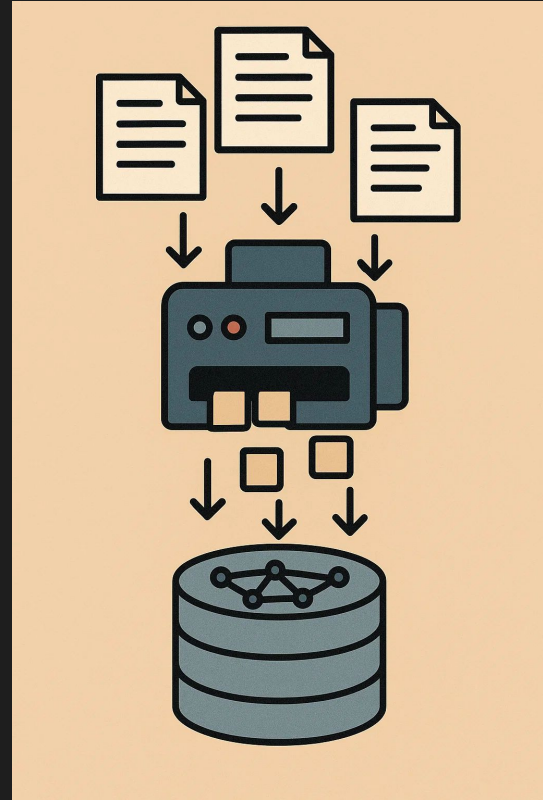
LLM processes this combined information to generate an improved response.



RAG Components: 1 The Knowledge Base & Data Prep

The foundation: Repository of external information (text docs, enterprise data).

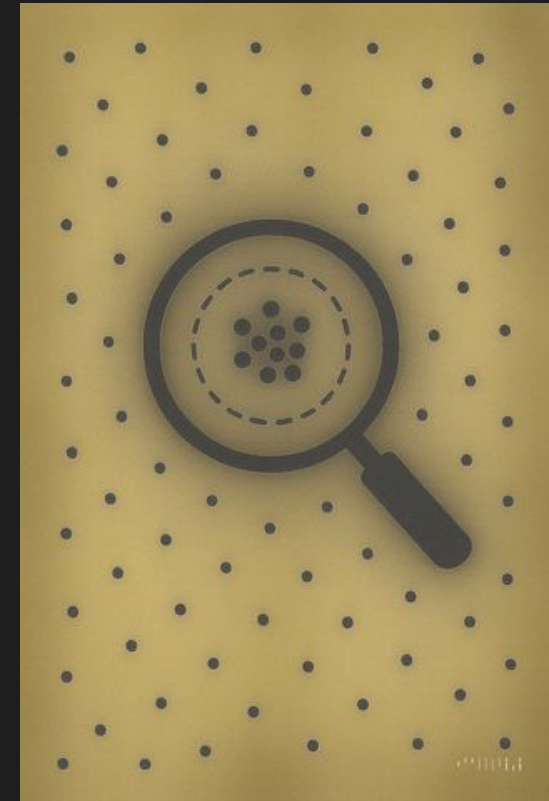
- **Data Sourcing:** Identifying relevant and authoritative data.
- **Chunking:** Breaking down large documents into smaller, manageable segments. Crucial for retrieval accuracy.
- **Embedding:** Converting text chunks into numerical vector representations using embedding models (captures semantic meaning).
- **Vector Store:** Storing these embeddings in a specialized vector database for efficient similarity search.



RAG Components: 2 The Retriever

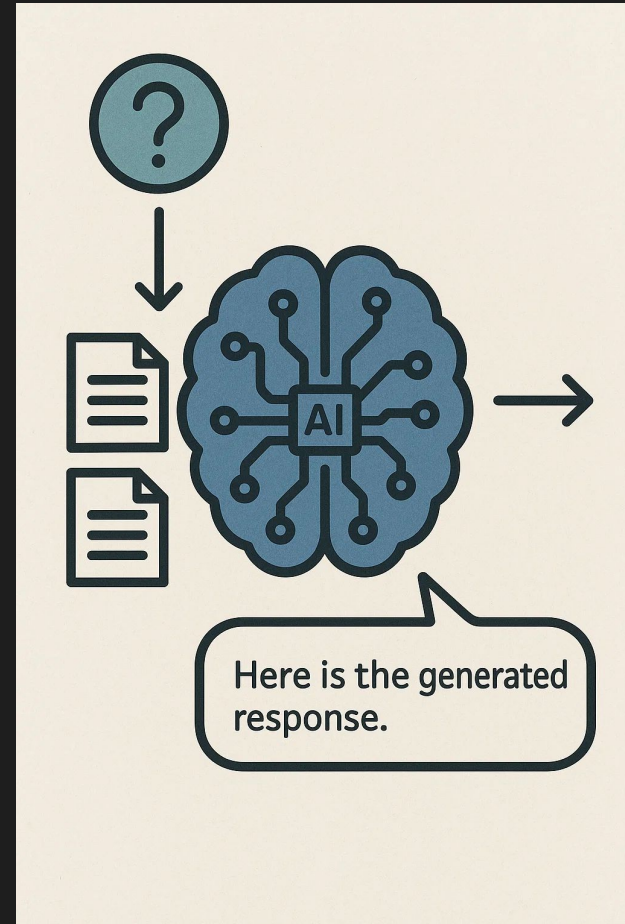
Fetches relevant context from the Knowledge Base based on the user query.

- **Query Embedding:** User query is converted into a vector using the *same* embedding model.
- **Similarity Search:** Compares the query vector to document vectors in the vector database (using metrics like cosine similarity, dot product, Euclidean distance).
- **Vector Databases:** Specialized DBs optimized for storing and querying high-dimensional vectors (e.g., FAISS, Chroma, Milvus, Pinecone, Weaviate).
- **Indexing:** Strategies (e.g., HNSW, IVF) used within vector DBs to speed up search.
- *(Optional: Mention Hybrid Search - combining*



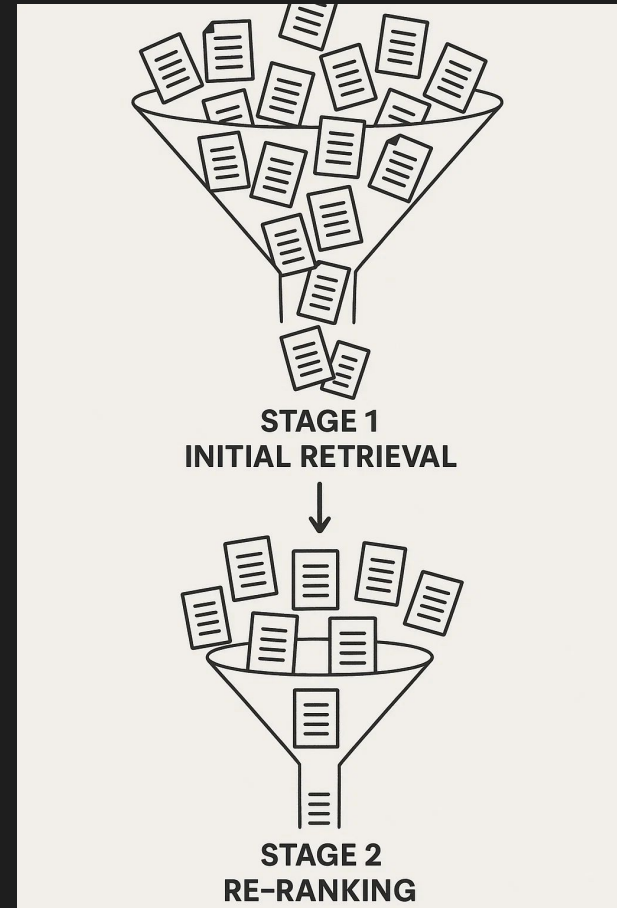
RAG Components: 3 The Generator (LLM)

- Typically a Large Language Model (LLM).
- Receives the augmented prompt (original query + retrieved context).
- Generates the final, context-aware response for the user.
- **Prompt Engineering:** Crucial for guiding the LLM to use the provided context effectively. The structure of the augmented prompt matters.



Improving Relevance: Ranking & Re-ranking

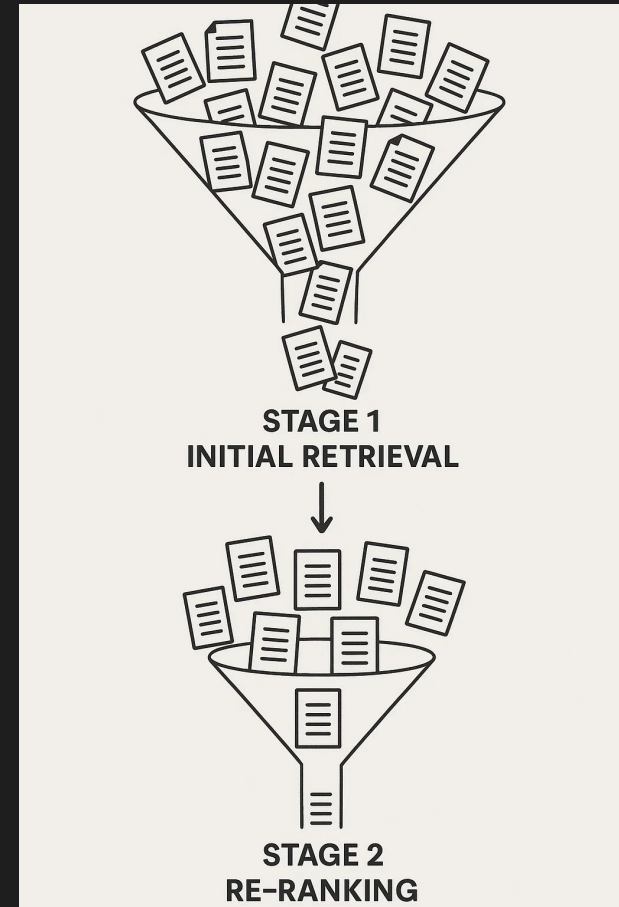
- Initial retrieval might return many documents; not all are equally relevant.
- **Ranking:** Ranking systems prioritize retrieved documents by relevance to the query. Order impacts LLM focus and response quality.
- **Vector Storage (Vector Databases):** Store vector embeddings. Designed for efficient similarity search. Examples (e.g., Milvus, Pinecone, Weaviate, Elasticsearch, Qdrant, Chroma)
- **Consistency in vector dimensionality is important.**



Improving Relevance: Ranking & Re-ranking

Retrieval Stage:

- Finds vectors most similar to the query embedding.
- **Similarity Metrics:** Cosine similarity, dot product, Euclidean distance.
- Hybrid Search: **Combining vector search with keyword methods (e.g., BM25) can improve recall and precision.**



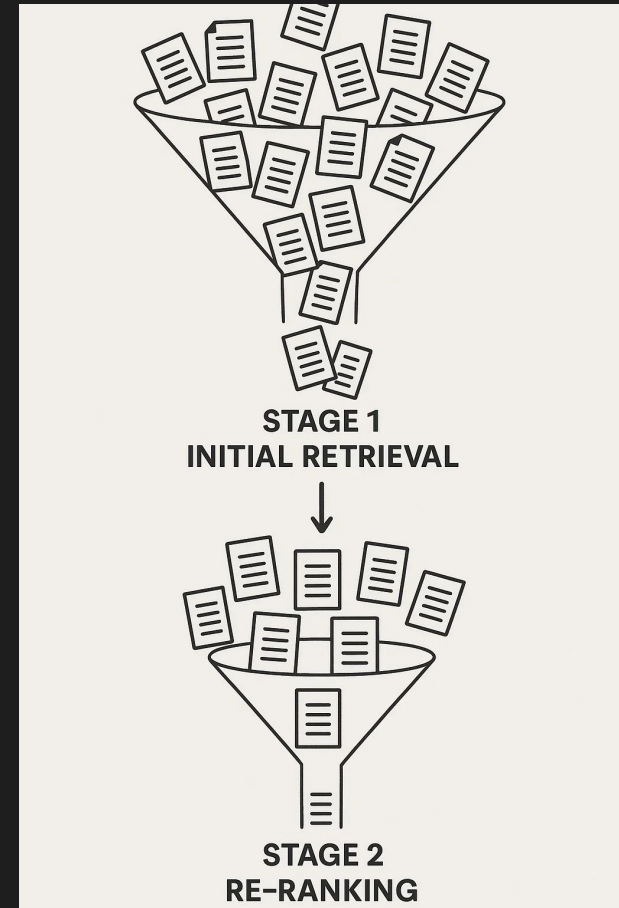
Improving Relevance: Ranking- Indexing

- **Purpose of Indexing:** Optimizes retrieval efficiency and scalability, especially for large knowledge bases.
- Reduces computational cost and latency.
- **Common Indexing Techniques:**
 - a. **HNSW (Hierarchical Navigable Small World):** Builds a multi-layer graph for efficient nearest neighbor searches.
 - b. **IVF (Inverted File Index):** Divides vectors into clusters to speed up search.
 - c. **PQ (Product Quantization):** Compresses vector data for reduced memory and efficient search.
 - d. (Briefly explain the core idea of one, e.g., HNSW as creating "express lanes" in a data highway).
- **Trade-offs:** Choice involves balancing search speed, accuracy, memory usage, and build time.



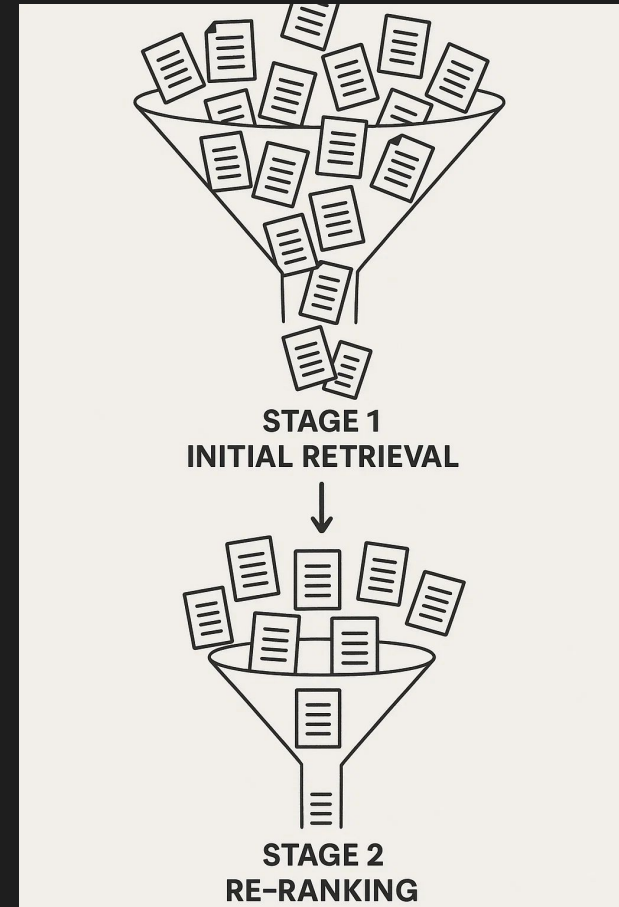
Improving Relevance: Ranking & Re-ranking

- Purpose: Re-evaluates and reorders initially retrieved documents based on semantic relevance to the query.
- Initial retrieval casts a wide net (recall-focused); re-ranking refines for precision.
- Ensures the LLM receives the most meaningful context.
- **How it Works:** A second-stage process using more advanced (often computationally intensive) models.
-



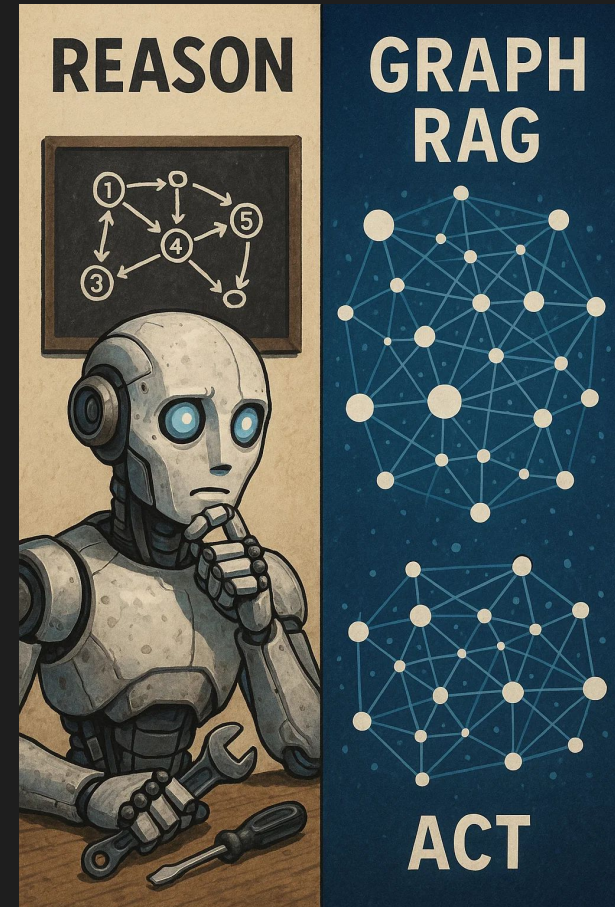
Improving Relevance: Ranking & Re-ranking

- **Types of Re-rankers:**
 - Cross-encoders:** Process query and document together for deep semantic understanding (e.g., BERT-based, MonoT5).
 - Multi-vector models (e.g., ColBERT):** Late interaction, efficient for large collections.
 - LLMs as Re-rankers (e.g., RankGPT):** Leverage LLM's language understanding to evaluate relevance.



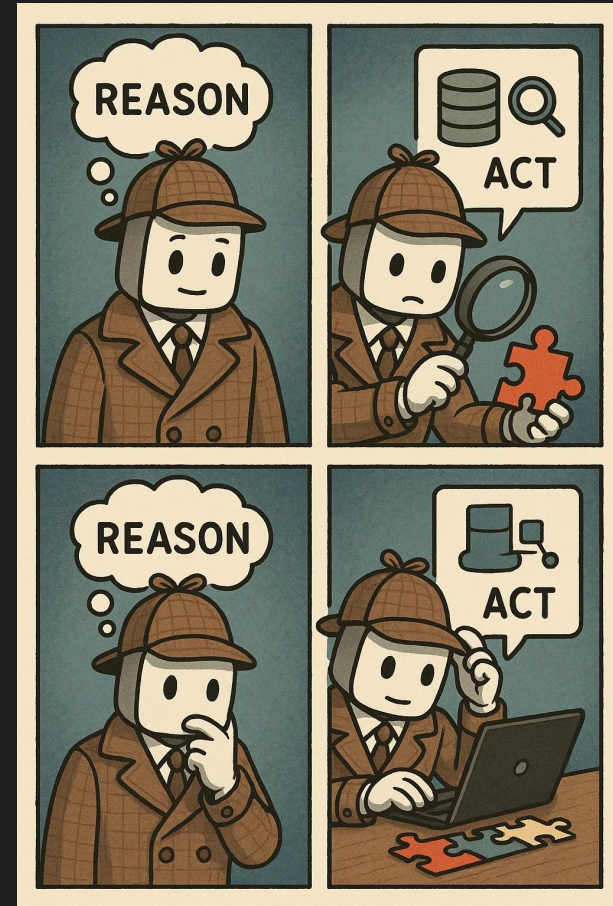
Beyond Basic RAG: Advanced Techniques

- **Agentic RAG (e.g., ReAct):** LLM acts like an agent, reasoning and deciding *what* information to retrieve, potentially in multiple steps, using tools. Breaks down complex queries.
- **Graph RAG:** Uses knowledge graphs instead of/alongside vector stores. Leverages relationships between entities for more structured retrieval. Good for complex queries involving connections.
- *Sentence Window Retrieval, Auto-merging Retrieval*



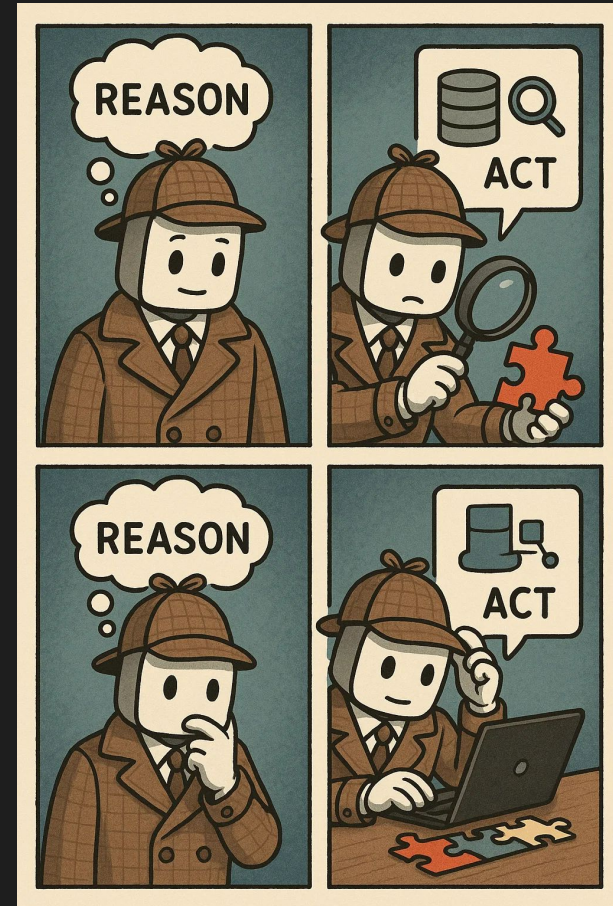
Enhancing RAG with ReAct (Reason + Act)

- **ReAct (Reasoning and Acting):** Allows LLMs to generate reasoning traces and task-specific actions in an interleaved manner.
- **How it Works:**
 - Reasoning:** Helps model induce, track, and update action plans; handle exceptions.
 - Acting:** Enables LLM to interface with external tools/knowledge bases to gather information.
- **Benefits for RAG:**
 - Decomposes complex queries into smaller, manageable steps.
 - Performs targeted retrieval for each step, using results to guide subsequent actions.
 - Reduces hallucinations by grounding reasoning in retrieved facts.
 - Handles multi-faceted and procedural queries more effectively.



Enhancing RAG with ReAct (Reason + Act)

- **Example (Medical Query):** "What are causes of fever and joint pain, and treatment options?"
 - a. ReAct agent reasons: two parts (causes, treatments).
 - b. Action 1: Retrieve causes.
 - c. Reason: Based on causes, investigate specific conditions.
 - d. Action 2: Retrieve treatments for identified conditions.
 - e. Synthesize comprehensive answer.



Summary & Transition to Demos

- RAG significantly improves LLM accuracy and relevance by grounding responses in external data.
- Core components: Knowledge Base (Vectors), Retriever (Similarity Search), Generator (LLM).
- Techniques like Re-ranking, Agentic RAG, and Graph RAG offer further enhancements.
- Now, let's see how to implement these concepts in practice using Google Colab!

The Path Forward

- **Future Trends:**
 - Standardization:** Simpler implementation and deployment of RAG solutions.
 - Advanced Agent-based RAG:** Greater flexibility and reasoning.
 - Multimodal RAG:** Handling images, audio, video alongside text.
 - Optimized Techniques:** Ongoing research in indexing, retrieval, and re-ranking.
 - Graph RAG:** Leveraging structured knowledge in graphs for more nuanced retrieval.



Assignment 1: Simple RAG with LlamaIndex

1. **Installations:** `llama-index`, `pypdf`, `sentence-transformers`, `torch`, `accelerate` (needed for some HF models).
2. **Imports:** Necessary classes from `llama_index.core`, `llama_index.readers.file`, `llama_index.embeddings.huggingface`, `llama_index.llms.huggingface`.
3. **Setup LLM & Embedding Model:**
 - a. Load a `SentenceTransformer` model for embeddings (e.g., `all-MiniLM-L6-v2`).
 - b. Load a Hugging Face LLM for generation (e.g., `google/flan-t5-small` or `google/flan-t5-base`). Configure it using `llama_index.llms.huggingface`. *Note: Mention larger models might need more RAM/GPU.*
 - c. Set up `Settings` in `LlamaIndex` to use these models globally.
4. **Load Data:** Use `SimpleDirectoryReader` pointing to the `pdfs` folder to load documents.
5. **Build Index (Chunking, Embedding, Storing):**
 - a. Use `VectorStoreIndex.from_documents()` – `LlamaIndex` handles chunking, embedding (using the configured model), and storing in an in-memory vector store automatically.
6. **Create Query Engine:** Get a query engine from the index: `query_engine = index.as_query_engine()`.
7. **Query:** Define a question related to the PDF content and run `response = query_engine.query("Your question here")`.
8. **Display Response:** Print the `response.response` and optionally `response.source_nodes` to see retrieved chunks.
9. **Markdown:** Explain each step, especially the abstraction provided by `LlamaIndex`.